

ПРАКТИЧЕСКАЯ РАБОТА № 10

Тема: Анализ защищенного протокола SSL\TLS

Программное обеспечение:

1. Любая операционная система (можно использовать физическую или виртуальную среду)
2. Wireshark (находится в папке PRZ_10_distr)
<https://drive.google.com/drive/folders/1MTpue1F4Nl6VIPMcUYyqsMMIFq1niQJm?usp=sharing>
3. Firefox (находится в папке PRZ_10_distr)
<https://drive.google.com/drive/folders/1MTpue1F4Nl6VIPMcUYyqsMMIFq1niQJm?usp=sharing>

Дистрибутивы, находящиеся в папке PRZ_10_distr запускаются с помощью виртуальной машины.

Вопросы для обсуждения:

1. Что такое SSL и TLS? В чём разница между ними?
2. Какие основные цели обеспечивает протокол TLS?
3. Опишите этапы установления TLS-соединения (TLS handshake).
4. Какие алгоритмы шифрования используются в TLS?
5. Что такое сертификат SSL/TLS и как он устроен?
6. Как работает асимметричная и симметричная криптография в TLS?
7. Что такое CA (Certificate Authority) и какую роль она играет в TLS?
8. Что означает термин "Forward Secrecy" в контексте TLS?
9. Какие версии протокола TLS считаются безопасными на текущий момент?
10. Как с помощью Wireshark проанализировать TLS-трафик?

Цель: Получение теоретических и практических навыков проведения анализа защищенного протокола SSL\TLS

Основные теоретические сведения

1. Общая характеристика протокола TLS

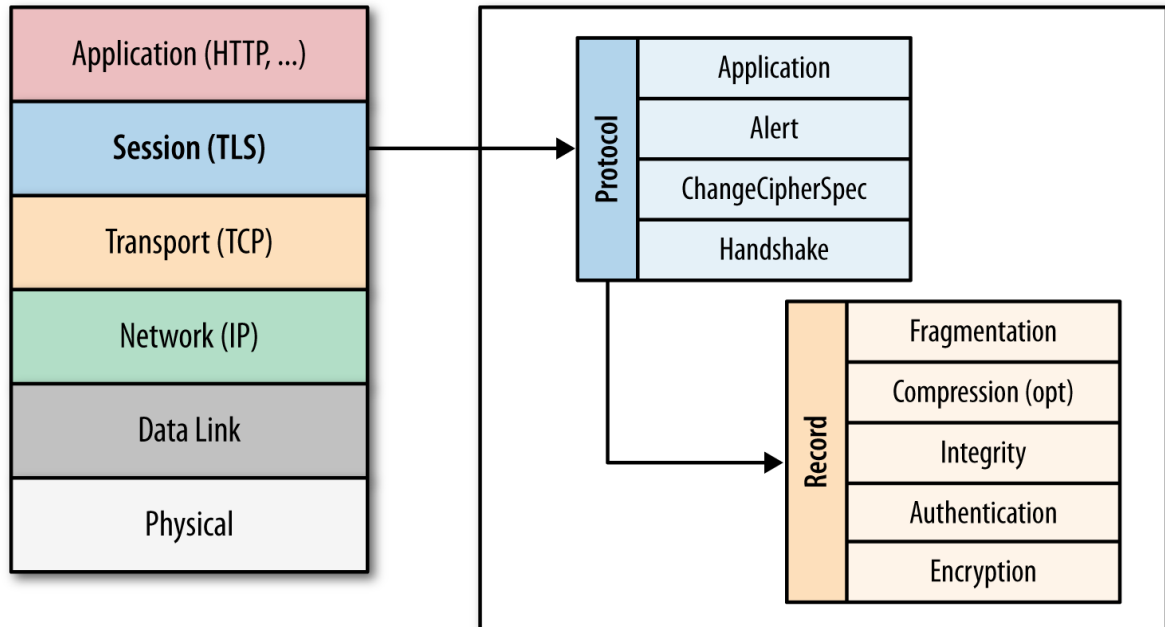
Протокол TLS (transport layer security) основан на протоколе SSL (Secure Sockets Layer), изначально разработанном в Netscape для повышения безопасности электронной коммерции в Интернете. Протокол SSL был реализован на application-уровне, непосредственно над TCP (Transmission Control Protocol), что позволяет более высокоуровневым протоколам (таким как HTTP или протоколу электронной почты) работать без изменений. Если SSL сконфигурирован корректно, то сторонний наблюдатель может узнать лишь параметры соединения (например, тип используемого шифрования), а также частоту пересылки и примерное количество данных, но не может читать и изменять их.

TLS используется для обеспечения безопасности целого ряда прикладных протоколов:

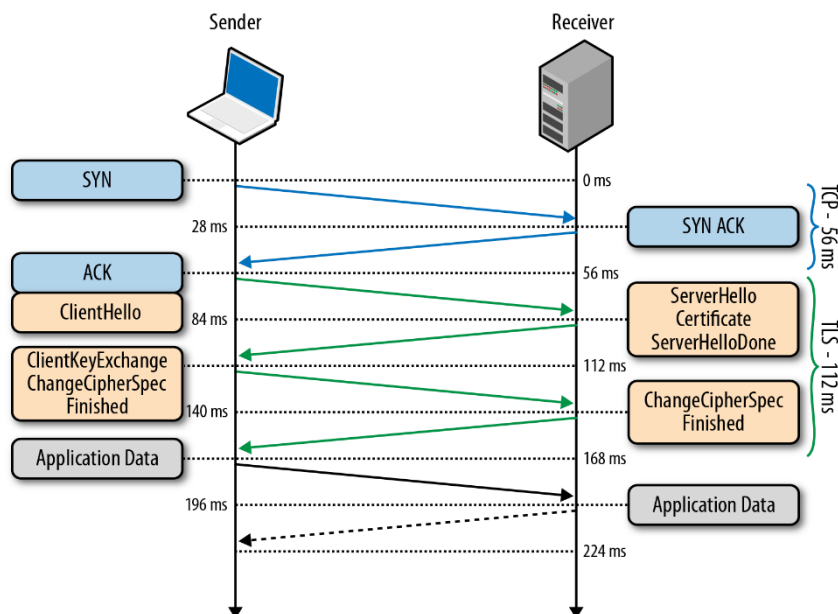
Протокол	Порт	Описание
HTTPS	443	HTTP по SSL/TLS
SMTPS	465	SMTP (электронная почта) по SSL/TLS
NNTPS	563	NNTP (новости) по SSL/TLS
LDAPS	636	LDAP (доступ к каталогам) по SSL/TLS
POP3S	995	POP (электронная почта) по SSL/TLS
IRCS	994	IRC (чат) по SSL/TLS
IMAPS	993	IMAP (электронная почта) по SSL/TLS
FTPS	990	FTP (передача файлов) по SSL/TLS

После того, как протокол SSL был стандартизирован IETF (Internet Engineering Task Force), он был переименован в TLS. Первая выпущенная версия протокола имела название SSL 2.0, но была довольно быстро заменена на SSL 3.0 из-за обнаруженных уязвимостей. В январе 1999 года IETF открыто стандартизирует его под именем TLS 1.0. Затем в апреле 2006 года была опубликована версия TLS 1.1, которая расширяла первоначальные

возможности протокола и закрывала новые известные уязвимости. Актуальная версия протокола на данный момент – TLS 1.2, выпущенная в августе 2008 года. Конкретное место TLS (SSL) в стеке протоколов Интернета показано на схеме:



Общая схема установления соединения с использованием TLS имеет вид:

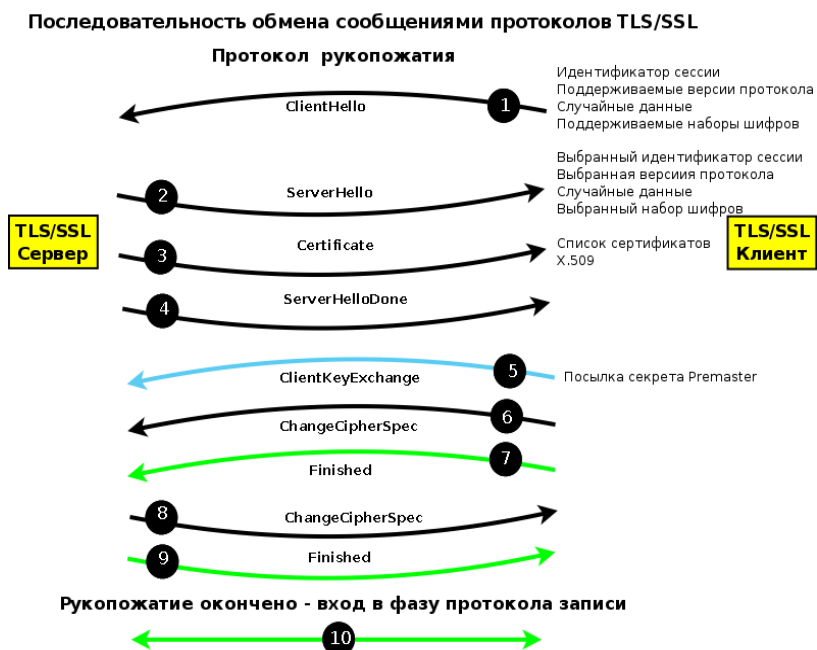


Временные издержки и ускорение реализации TLS.

Значения временных задержек зависят от типа оборудования, версии и реализации TLS. Эти временные затраты связаны с тем, что перед тем, как начать обмен данными через TLS, клиент и сервер должны согласовать параметры соединения: версия используемого протокола, способ шифрования

данных, а также проверить сертификаты, если это необходимо. Отметим SSL/TLS в Windows 2000 работает немного быстрее, если используется 128-, а не 40- или 56-разрядный ключ, а при увеличении длины ключа с 512 до 1024 бит количество соединений в секунду резко падает и может уменьшиться в пять раз.

Более детальная схема начала установки соединения (TLS Handshake) показана на схеме:



Когда версия протокола и способ шифрования считаются утвержденными, клиент проверяет присланный сертификат и инициирует обмен ключами либо на основе RSA, либо по Диффи-Хеллману, в зависимости от установленных параметров.

По различным историческим и коммерческим причинам чаще всего в TLS используется обмен ключами по алгоритму RSA: клиент генерирует симметричный ключ, подписывает его, шифрует с помощью открытого ключа сервера и отправляет его на сервер. Обмен ключами Диффи-Хеллмана представляется более защищенным, так как установленный симметричный ключ никогда не покидает клиента или сервера и, соответственно, не может быть перехвачен злоумышленником, даже если тот знает закрытый ключ сервера.

Производительность программных реализаций алгоритма RSA очень низка и очень быстро снижается при увеличении длины ключа.

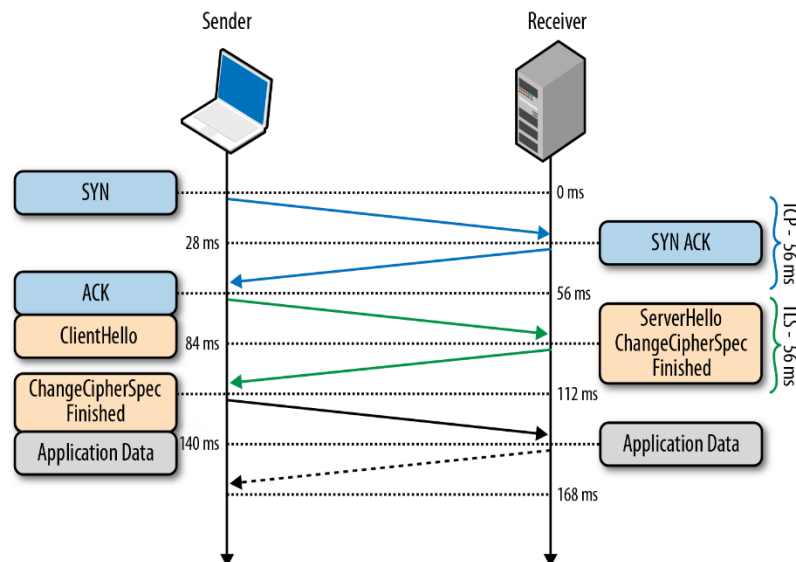
Начиная с первой публичной версии протокола (SSL 2.0) сервер в рамках TLS Handshake (а именно первоначального сообщения ServerHello) может сгенерировать и отправить 32-байтный идентификатор сессии (connection-id). Начальный обмен сообщениями:

Client-hello	C ® S:	challenge, cipher_specs
Server-hello	S ® C:	connection-id, server_certificate, cipher_specs
Client-master-key	C ® S:	{master_key}server_public_key
Client-finish	C ® S:	{connection-id}client_write_key
Server-verify	S ® C:	{challenge}server_write_key
Server-finish	S ® C:	{new_session_id}server_write_key

Если у сервера хранится кэш сгенерированных идентификаторов и параметров сеанса для каждого клиента, то в свою очередь клиент хранит у себя присланный идентификатор и включает его (конечно, если он есть) в первоначальное сообщение ClientHello. Если и клиент, и сервер имеют идентичные идентификаторы сессии, то установка общего соединения происходит по упрощенному алгоритму:

Client-hello	C ® S:	challenge, session_id, cipher_specs
Server-hello	S ® C:	connection-id, session_id_hit
Client-finish	C ® S:	{connection-id}client_write_key
Server-verify	S ® C:	{challenge}server_write_key
Server-finish	S ® C:	{session_id}server_write_key

Процедура возобновления сессии позволяет пропустить этап генерации симметричного ключа, что существенно повышает время установки соединения, но не влияет на его безопасность, так как используются ранее не скомпрометированные данные предыдущей сессии.

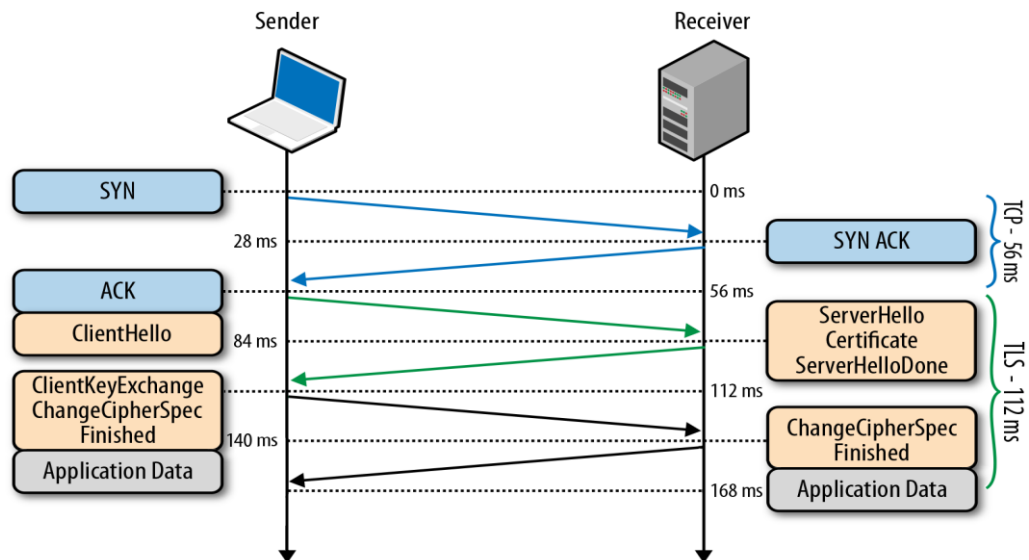


Считается, что установка нового SSL/TLS-соединения занимает приблизительно в пять раз больше времени, чем восстановление соединения, помещенного в кэш. Стандартный тайм-аут для такого соединения в Windows NT 4 увеличен с 2 до 5 минут. Время жизни соединения в кэше можно увеличить, но при слишком большом таймауте система будет тратить память на кэширование устаревших соединений.

Технология возобновления сессии бесспорно повышает производительность протокола и снижает вычислительные затраты, однако она не применима в первоначальном соединении с сервером, или в случае, когда предыдущая сессия уже истекла.

Для получения ещё большего быстродействия была разработана технология TLS False Start, являющаяся опциональным расширением протокола и позволяющая отправлять данные, когда TLS Handshake завершён лишь частично. Важно отметить, что TLS False Start никак не изменяет процедуру TLS Handshake. Он основан на предположении, что в тот момент, когда клиент и сервер уже знают о параметрах соединения и симметричных ключах, данные приложений уже могут быть отправлены, а все необходимые проверки можно провести параллельно. В результате соединение готово к использованию на одну итерацию обмена сообщениями раньше.

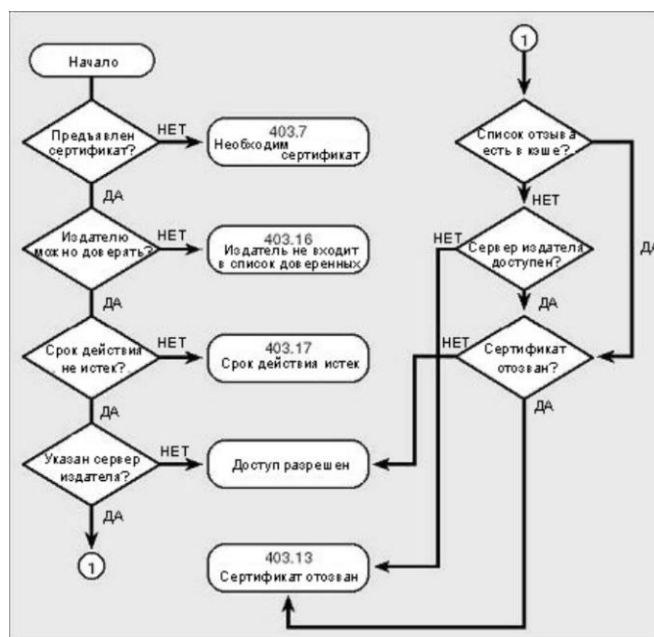
Процедура TLS False Start представлена на схеме:



Аутентификация в TLS

Содержание самой аутентификации предполагает выполнение ряда процедур сервером\клиентом:

- выполняется криптографическая проверка наличия у сервера личного ключа, соответствующего открытому ключу, указанному в сертификате;
- проверяется степень доверия издателю сертификата;
- проверяется, не истек ли срок действия сертификата;
- проверяется, не отозван ли сертификат.



В настоящее время в качестве сертификата предлагается общий стандарт для Интернет с использованием формата X.509. Сертификат открытого ключа

удостоверяет принадлежность открытого ключа некоторому субъекту, например, пользователю. Сертификат открытого ключа содержит имя субъекта, открытый ключ, имя удостоверяющего центра, политику использования соответствующего удостоверяемому открытому ключу закрытого ключа и другие параметры, заверенные подписью удостоверяющего центра. Формат сертификата открытого ключа X.509 v3 описан в RFC 5280.

Версия	Версия v1	Версия v2	Версия v3
Серийный номер			
Идентификатор алгоритма подписи			
Имя издателя			
Период действия (не ранее / не позднее)			
Имя субъекта			
Информация об открытом ключе субъекта	Все версии	Все версии	Все версии
Уникальный идентификатор издателя			
Уникальный идентификатор субъекта			
Дополнения			
Подпись	Все версии		

Главная цель процесса обмена ключами — это создание секрета клиента (pre_master_secret), известного только клиенту и серверу. Секрет (pre_master_secret) используется для создания общего секрета (master_secret).

Общий секрет необходим для того, чтобы создать:

- сообщение для проверки сертификата,
- проверки ключей шифрования,
- секрета MAC (message authentication code),
- сообщения «finished».

Отсылая сообщение «finished», стороны указывают, что они знают верный секрет (pre_master_secret).

Отметим, что аутентификация опционально может быть односторонней (аутентификация сервера клиентом), взаимной (встречная аутентификация сервера и клиента) или не использоваться (в отличие от многих других реализаций TLS, например в HTTPS, большинство реализаций EAP-TLS требует предустановленного сертификата X.509 у клиента, не давая

возможности отключить это требование, хотя стандарт RFC 5216 не требует этого в обязательном порядке – одна из причин высокой защищенности метода EAP-TLS).

Безопасность протокола TLS.

Вся история развития семейства протоколов SSL-TSL связана с обнаружением уязвимостей протокола и их устранением в следующей версии. В связи с этим активизирован переход с SSL на TLS и на TLS последних версий, хотя по некоторым данным до 30% серверов и клиентов на старом ПО используют SSL, также отмечаются ошибки в настройках и даже “принудительное ослабление” TLS для уменьшения временных издержек.

Атаки можно условно разделить на несколько групп:

- атаки на протокол TLS, используемый для приложений сети интернет;
- атаки на ошибки в спецификациях протокола TLS;
- атаки на слабости используемых математических методов (генераторы случайных чисел, хэш-функции и криптографические алгоритмы);
- атаки на слабости реализации используемых математических методов.

Из приведенного ниже краткого обзора можно сделать вывод, что именно особенности установленного SSL\TLS соединения позволяют использовать тот или иной вид атаки и для подготовки атаки необходима тщательная предварительная разведка.

Атака Beast

Атака проводится многоэтапно. Для успешного осуществления атаки требуется запуск JavaScript-кода в браузере клиента, который должен быть запущен после установки SSL-соединения браузера с сайтом, данные которого требуется перехватить (SSL-соединение остается открытым длительное время, поэтому у атакующего есть запас времени). JavaScript-код не требуется запускать в контексте атакуемого сайта, его достаточно открыть в новой

вкладке, например, путем встраивания через `iframe` в какой-нибудь сторонний сайт, открытие которого можно стимулировать методами социальной инженерии. JavaScript-код используется для отправки на сайт, с которым работает жертва, фиктивных запросов с изначально известными контрольными метками, которые используются атакующим для воссоздания отдельных блоков для шифра TLS/SSL, работающего на базе алгоритма AES.

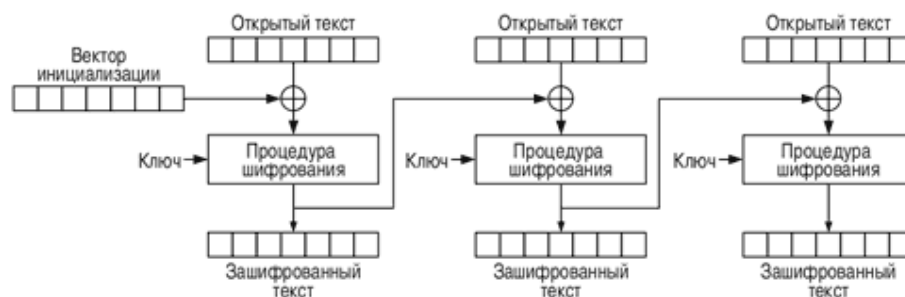
После того, как вспомогательный JavaScript-код запущен, на одном из промежуточных шлюзов осуществляется запуск сниффера, для выявления связанных с определенным сайтом TLS-соединений, их перехват и расшифровка начальной части HTTPS-запроса, в которой содержится HTTPS Cookie (запросы вспомогательного скрипта отправляются через ранее установленный SSL-канал, при этом браузер добавит к этим запросам все ранее установленные Cookie). После того, как удалось воссоздать идентифицирующую HTTPS Cookie, осуществляется классическая атака, позволяющая вклиниться в активную сессию жертвы. Расшифровка основана на методе угадывания содержимого отдельных блоков, часть которых содержит данные отправляемые подставным JavaScript-кодом.

Атака Crime

(Compression Ratio Info-leak Made Easy). Атака эффективна для каналов связи на основе SSL/TLS и SPDY только при использовании сжатия передаваемых данных и требует выполнения JavaScript-кода злоумышленника в браузере клиента. По своей сути Crime является продолжением развития идей атаки Beast. Атака строится на возможности выделения в отслеживаемом зашифрованном трафике блоков данных с метками, отправляемыми подставным JavaScript-кодом на сайт, для которого требуется перехватить Cookie, в рамках общего шифрованного канала связи. Используя тот факт, что данные сжимаются на этапе до шифрования и не подвергаются дополнительной рандомизации, новая атака позволяет обойти средства защиты, добавленные производителями браузеров для блокирования атаки Beast.

Padding Oracle Attack

Используется для атаки на использование симметричного блочного шифрования с режимом сцепления блоков шифротекста (Cipher Block Chaining) – один из режимов шифрования для симметричного блочного шифра с использованием механизма обратной связи. Каждый блок открытого текста (кроме первого) побитно складывается по модулю 2 (операция XOR) с предыдущим результатом шифрования.



Важно, что предпочтительным методом дополнения блоков шифротекста, является PKCS7 (есть другие варианты, например PKCS5, но для них так же возможно организация подобной атаки). В нем значение каждого дополняемого байта устанавливается равным количеству дополняемых байт. А знание факта, получается ли при расшифровке шифротекста открытый текст с корректным дополнением, достаточно атакующему для проведения успешной атаки на шифрование в режиме CBC. Если мы можем подавать какому-то сервису шифротексты, а он будет возвращать нам информацию, корректно ли дополнение (перехват ответа об ошибке) – мы сможем вскрыть любой шифротекст. Ruby OpenSSL подвержен данной проблеме.

Lucky 13

Уязвимость связана именно с недоработкой в спецификациях TLS, а не с конкретными реализациями, поэтому баг присутствует во всех библиотеках. Уязвимость, которая потенциально позволяет извлечь конфиденциальную информацию из зашифрованного канала связи, в том числе пароли и аутентификационные cookies, использующего режим сцепления блоков

шифротекста (CBC). Этот метод использует атаку типа padding oracle, подменяя шифротекст и анализируя временную задержку при ответе для определения ошибки при дополнении блока шифротекста.

Атака FREAK

Суть атаки сводится к инициированию отката соединения на использование разрешенного для экспорта набора шифров, включающего недостаточно защищенные устаревшие алгоритмы шифрования. Проблема позволяет вклиниться в соединение и организовать анализ трафика в рамках защищенного канала связи, используя уязвимость, выявленную во многих SSL-клиентах и позволяющую сменить шифры RSA на RSA_EXPORT и выполнить дешифровку трафика, воспользовавшись слабым эфемерным ключом RSA (512 бит). Для подбора ключа RSA-512 исследователям потребовалось около семи с половиной часов. На стороне клиента уязвимость затрагивает OpenSSL (исправлено в 0.9.8zd, 1.0.0p и 1.0.1k), браузер Safari и разнообразные встраиваемые и мобильные системы, включая Google Android и Apple iOS.

Атака Poodle

Padding Oracle On Downgraded Legacy Encryption. Атака позволяет восстановить содержимое отдельных секретных идентификаторов, передаваемых внутри зашифрованного SSLv3-соединения, и по своей сути напоминает такие ранее известные виды атак на HTTPS, как BREACH, CRIME и BEAST, но значительно проще для эксплуатации и не требует выполнения каких-то особых условий. Проблеме подвержен любой сайт, допускающий установку защищенных соединений с использованием протокола SSLv3, даже если в качестве более приоритетного протокола указаны актуальные версии TLS. Для отката на SSLv3 атакующие могут воспользоваться особенностью современных браузеров переходить на более низкую версию протокола, в случае сбоя установки соединения. Атака строится на возможности выделения в отслеживаемом зашифрованном трафике блоков данных с метками, отправляемыми подставным JavaScript-кодом на сайт, для которого требуется

перехватить идентификационные данные, в рамках общего зашифрованного канала связи.

Атака Logjam.

Это атака на TLS, которой подвержено большое число клиентских и серверных систем, использующих HTTPS, SSH, IPsec, SMTPS и другие протоколы на базе TLS. По своей сути Logjam напоминает представленную в марте атаку FREAK и отличается тем, что вместо инициирования смены шифров RSA на RSA_EXPORT в Logjam производится откат протокола Диффи-Хеллмана (Diffie-Hellman), используемого для получения ключа для дальнейшего шифрования, до слабозащищённого уровня DHE_EXPORT, что в сочетании с использованием не уникальных начальных простых чисел позволяет применить методы подбора ключа.

Взлом TLS и SSL через уязвимость в шифре RC4

Алгоритм RC4, как и любой потоковый шифр, строится на основе параметризованного ключом генератора псевдослучайных битов с равномерным распределением. Длина ключа может составлять от 40 до 256 бит. Основные преимущества шифра — высокая скорость работы и переменный размер ключа.

Уязвимость RC4 связана с недостаточной случайностью потока битов, которым скремблируется сообщение. Прогоняя через него одно сообщение много раз, удалось выявить достаточное количество повторяющихся паттернов для восстановления исходного сообщения. Однако для атаки нужно прогнать через шифр большой объем данных.

Новая MITM атака

Для осуществления атаки злоумышленнику необходимо каким-либо образом убедить жертву установить специально изготовленный клиентский SSL-сертификат, ключ от которого известен атакующему. В дальнейшем это позволяет «убедить» клиентский софт в том, что он взаимодействует с доверенным сервером, в то время как на самом деле коммуникация осуществляется с атакующим. Реализация TLS в библиотеке BouncyCastle

подвержена новой атаке. Кроме того, уязвима и TLS-библиотека операционной системы Mac OS X (Secure Transport).

Слабости ключей “малой” размерности

Необходимым условием применения TLS в сетях с произвольными, меняющимися парами соединений является использование криптографии с открытым ключом. Однако в отличие от классической симметричной криптографии стойкость этих методов основывается на недоказанной пока сложности решения ряда задач (факторизация простых чисел и логарифмирование над конечным полем). Кроме того, ввиду вычислительной сложности этих задач при реализации они ограничиваются практически соображениями по определению размеров ключевой информации. А вычислительные возможности атак на такие алгоритмы постоянно возрастают.

Еще в 1990-х годах был разработан алгоритм факторизации целых чисел под названием “метод решета числового поля”. Для криптографии он был интересен тем, что позволил успешно получить доступ к данным, зашифрованным при помощи 768-битного ключа RSA.

В 2015 году опубликована статья, в которой NFS-метод, использующий результаты предвычислений, был успешно применен для решения задачи дискретного логарифмирования и атаки на процедуру Диффи-Хеллмана.

NFS-метод дает возможность для любого простого числа p провести процедуру предвычислений, позволяющую в дальнейшем быстро получать любой конкретный дискретный логарифм (то есть решать относительно x уравнение $g^x = y \bmod p$ для любого элемента y фиксированной мультипликативной группы конечного поля вычетов). Задача упрощается тем, что для популярных интернет-протоколов (на текущий момент на них функционируют миллионы серверов) используются одни и те же группы (во многих реализациях простые числа либо защиты в код, либо постоянно используются одни и те же наборы рекомендованных “безопасных” простых чисел).

Слабости генераторов случайных чисел.

Качественные случайные и псевдослучайные последовательности играют огромную роль при использовании криптографических средств, и зачастую от них полностью зависит стойкость системы. Можно привести следующие примеры случаев, в которых использование плохого источника случайности приводит к краху безопасности всей системы.

Использование одного и того же значения в качестве случайного параметра в алгоритмах подписи DSA, ECDSA, ГОСТ Р 34.10-94, ГОСТ Р 34.10-2001, ГОСТ Р 34.10-2012 при обработке двух разных сообщений на одном ключе позволяет восстановить секретные ключи подписи и подделывать подпись. Подобная ошибка в реализации ECDSA была успешно использована при обходе защиты в Sony PlayStation 3.

Использование одного и того же случайного набора в качестве синхропосылки (IV) поточных шифров или блочных шифров, работающих в режимах CFB или гаммирования, приводит к перекрытию гаммы и к возможности восстановления частей открытого текста. Подобное уже случалось в реальных системах – неправильное использование шифра RC4 позволяло восстановить части документов MS Word и Excel по последовательности автосохраненных копий.

Нарушение случайности генерации простых чисел для схемы RSA.

Кроме ошибок возможны и «закладки» - «ошибки» в алгоритмах ряда генераторов: Dual_EC_DRBG, Intel Secure Key.

Приведенный выше неполный список “свежих” уязвимостей показывает, что атаки направлены на вскрытие криптографических протоколов, но используют для этого «слабости» протокола TLS. В тоже время эти слабости связаны с использованием TLS для HTTPS. Основной уязвимостью в этом случае является некорректное применение и реализация криптографических методов, что позволяет проводить на них атаки по известным фрагментам шифротекста, использовать криптозакладки или «ослаблять» стойкость применяемых шрифтов.

Кроме того, практически все атаки носят многоэтапный характер, где, на

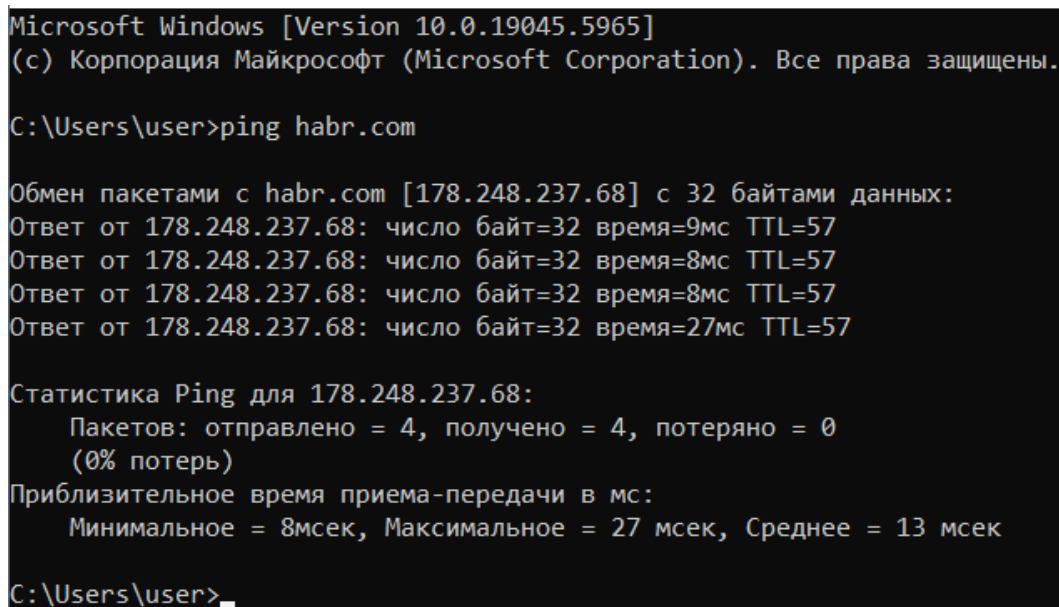
основе атак по перехвату трафика и атаки MITM, возникает возможность использовать слабости протокола.

В качестве общих рекомендаций можно предложить использование TLS последних версий, тщательную настройку параметров протоколов, максимальное увеличение размера ключей в системах с открытым ключом. И как было указано выше из приведенного краткого обзора видно, что именно особенности установленного SSL\TLS соединения позволяют использовать тот или иной вид атаки, следовательно для подготовки атаки необходима тщательная предварительная разведка.

2. Пример анализа TLS соединения в Wireshark

Ниже приведен пример анализа TLS соединения с сервером <https://habr.com>.

1) habr.com ip-address 178.248.237.68



```
Microsoft Windows [Version 10.0.19045.5965]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

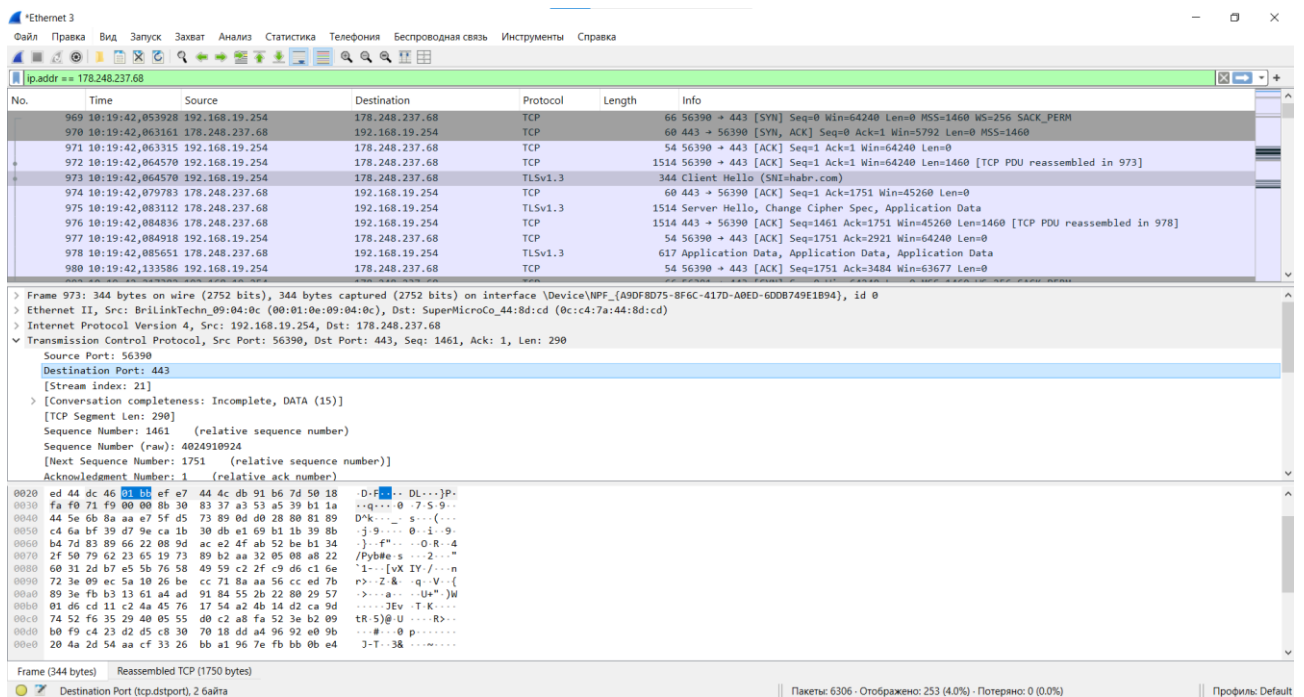
C:\Users\user>ping habr.com

Обмен пакетами с habr.com [178.248.237.68] с 32 байтами данных:
Ответ от 178.248.237.68: число байт=32 время=9мс TTL=57
Ответ от 178.248.237.68: число байт=32 время=8мс TTL=57
Ответ от 178.248.237.68: число байт=32 время=8мс TTL=57
Ответ от 178.248.237.68: число байт=32 время=27мс TTL=57

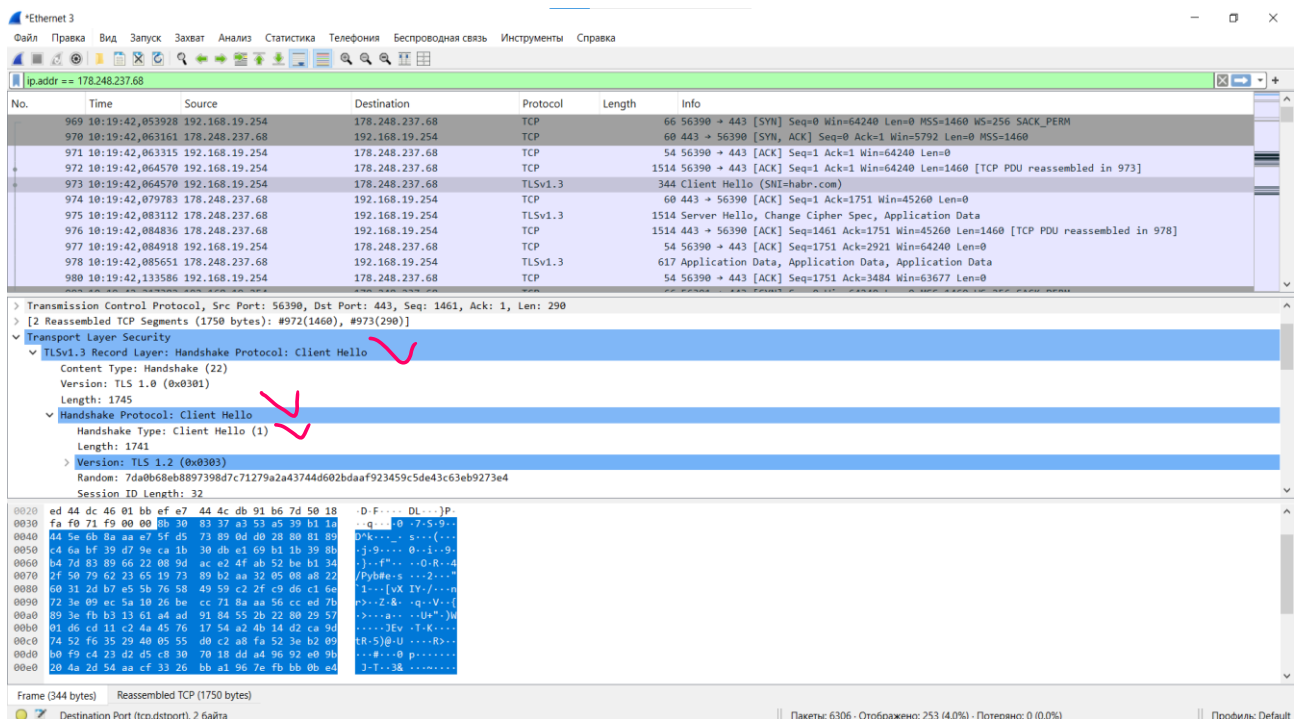
Статистика Ping для 178.248.237.68:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 8мсек, Максимальное = 27 мсек, Среднее = 13 мсек

C:\Users\user>
```

2) При соединении с habr.com подключение происходит по 443 порту, что представлено на рисунке ниже.



3) Приветствие клиента



первый байт пакета в HEX равен $0x16 = 22$, что означает, что «запись» является «рукопожатием». Следующие два байта — $0x0303$, означающие версию 3.3, что говорит о том, что TLS 1.2 на самом деле SSL 3.3. Запись с рукопожатием разбита на несколько сообщений. Первый — «приветствие клиента» (0x01). Здесь есть несколько важных моментов:

3.1) Случайность:

972	10:19:42,064570	192.168.19.254	178.248.237.68	TCP	1514 56390 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=1460 [TCP PDU reassembled in 973]
973	10:19:42,064570	192.168.19.254	178.248.237.68	TLSv1.3	344 Client Hello (SHA-habr.com)
974	10:19:42,079783	178.248.237.68	192.168.19.254	TCP	60 443 → 56390 [ACK] Seq=1 Ack=1751 Win=45260 Len=0
975	10:19:42,083112	178.248.237.68	192.168.19.254	TLSv1.3	1514 Server Hello, Change Cipher Spec, Application Data
976	10:19:42,084836	178.248.237.68	192.168.19.254	TCP	1514 443 → 56390 [ACK] Seq=1461 Ack=1751 Win=45260 Len=1460 [TCP PDU reassembled in 978]
977	10:19:42,084918	192.168.19.254	178.248.237.68	TCP	54 56390 → 443 [ACK] Seq=1751 Ack=2921 Win=64240 Len=0
978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3	617 Application Data, Application Data, Application Data
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	54 56390 → 443 [ACK] Seq=1751 Ack=3484 Win=63677 Len=0

Length: 1745	
▼	Handshake Protocol: Client Hello
Handshake Type: Client Hello (1)	
Length: 1741	
▶	Version: TLS 1.2 (0x0303)
Random: 7da0b68eb8897398d7c71279a2a43744d602bdaaf923459c5de43c63eb9273e4	
Session ID Length: 32	
Session ID: d3028f7e4d8fc3b5471bd3283a9e2a83e96b9afe67bd632566ec4457afd73434	
Cipher Suites Length: 32	
▶	Cipher Suites (16 suites)

32 байта случайных значений - Client Random. В спецификации рекомендовалось использовать первые 4 байта для передачи UNIX-таймстемпа, а оставшиеся 28 - заполнять результатом работы криптографического генератора псевдослучайных чисел. Однако сейчас многие браузеры и веб-серверы генерируют все 32-байта случайным образом. Забегая чуть вперёд: изначально предполагалось, что наличие таймстемпа в handshake-сообщениях может помочь при обнаружении проблем с подменой времени на том или ином узле. Однако данный метод сейчас никак не используется, поэтому байты ClientRandom часто просто случайны, что и наблюдаем здесь.

3.2) ID сессии:

980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	
Length: 1741					
▶	Version: TLS 1.2 (0x0303)				
Random: 7da0b68eb8897398d7c71279a2a43744d602bdaaf923459c5de43c63eb9273e4					
Session ID Length: 32					
Session ID: d3028f7e4d8fc3b5471bd3283a9e2a83e96b9afe67bd632566ec4457afd73434					
Cipher Suites Length: 32					
▶	Cipher Suites (16 suites)				
Compression Methods Length: 1					
▶	Compression Methods (1 method)				
Extensions Length: 1636					
▶	Extension: Reserved (GREASE) (len=0)				
▶	Extension: application_settings (len=5)				
▶	Extension: supported_groups (len=12)				

0020	23	45	9c	5d	e4	3c	63	eb	92	73	e4	20	d3	02	8f	7e	#E.]<c.~s.~...
0030	4d	8f	c3	b5	47	1b	d3	28	3a	9e	2a	83	e9	6b	9a	fe	M...G.../..*.k..

идентификатор TLS-сессии - SessionID: TLS позволяет возобновлять ранее установленные сессии, используя сокращённый вариант протокола установления соединения. Идентификатор сессии как раз содержит номер такой сессии, параметры которой (возможно) сохранены на сервере. В данном

случае поле идентификатора сессии пусто.

3.3) Cipher Suites: список шифронаборов, которые поддерживает клиент - Cipher Suites. Порядок шифронаборов в списке отражает их степень предпочтения (предпочтительные идут первыми);

973	10:19:42,064570	192.168.19.254	178.248.237.68	TLSv1.3	344 Client Hello (SNI=habr.com)
974	10:19:42,079783	178.248.237.68	192.168.19.254	TCP	60 443 → 56390 [ACK] Seq=1 Ack=1751 Win=45260 Len=0
975	10:19:42,083112	178.248.237.68	192.168.19.254	TLSv1.3	1514 Server Hello, Change Cipher Spec, Application Data
976	10:19:42,084836	178.248.237.68	192.168.19.254	TCP	1514 443 → 56390 [ACK] Seq=1461 Ack=1751 Win=45260 Len=1460 [TCP PDU reassembled in 978]
977	10:19:42,084918	192.168.19.254	178.248.237.68	TCP	54 56390 → 443 [ACK] Seq=1751 Ack=2921 Win=64240 Len=0
978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3	617 Application Data, Application Data, Application Data
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	54 56390 → 443 [ACK] Seq=1751 Ack=3484 Win=63677 Len=0
Cipher Suites (16 suites)					
Cipher Suite: Reserved (GREASE) (0xeeaa)					
Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)					
Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)					
Cipher Suite: TLS_CHACHA20_POLY1305_SHA256 (0x1303)					
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)					
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)					
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c)					
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)					
Cipher Suite: TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca9)					
Cipher Suite: TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xc03a)					
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)					
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)					
0020	23 45 9c 5d e4 3c 63 eb	92 73 e4 20 d3 02 8f 7e	#E-]-<c- s- ...		
0030	4d 8f c3 b5 47 1b d3 28	3a 9e 2a 83 e9 6b 9a fe	M-..G-(: :F-..k-..		
0040	67 bd 63 25 66 ec 44 57	af d7 34 34 00 20 ea ea	g-cXf-DW-..44-..		
0050	13 01 13 02 13 03 c0 2b	c0 2f c0 2c 00 30 cc a9	+../.0-..		
0060	cc a8 c0 13 c0 14 00 9c	00 9d 00 2f 00 35 01 00	/.5-..		
0070	06 64 0a 0a 00 00 44 69	00 05 00 03 02 68 32 00	-d-...D1-...-h2-		

3.4) Расширение server_name:

976	10:19:42,084836	178.248.237.68	192.168.19.254	TCP	
977	10:19:42,084918	192.168.19.254	178.248.237.68	TCP	
978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3	
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	
Extension: compress_certificate (len=3)					
Extension: ec_point_formats (len=2)					
Extension: server_name (len=13) name=habr.com					
Type: server_name (0)					
Length: 13					
Server Name Indication extension					
Extension: renegotiation_info (len=1)					
Extension: application_layer_protocol_negotiation (len=14)					
Extension: extended_master_secret (len=0)					
Extension: signed_certificate_timestamp (len=0)					
Extension: psk_key_exchange_modes (len=2)					
Extension: key_share (len=1263) Unknown (4588), x25519					
Extension: Reserved (GREASE) (len=1)					
01a0	00 02 00 0b 00 02 01 00	00 00 00 0d 00 0b 00 00	-....		
01b0	08 68 61 62 72 2e 63 6f	6d ff 01 00 01 00 00 10	.habr.co m.....		
01c0	00 0e 00 0c 02 68 32 08	68 74 74 70 2f 31 2e 31	h2- http/1.1		
01d0	00 17 00 00 00 12 00 00	00 2d 00 02 01 01 00 33	-....3		
01e0	04 ef 04 ed ea ea 00 01	00 11 ec 04 c0 2c 4a 50	-....JP		
01f0	bd 91 1c bc 2b 84 d1 60	84 cf 14 a9 79 50 44 1b	+..`....yPD-		
0200	d9 52 b0 86 a5 b8 44 05	2d 11 48 e5 ec 6a 5d 7a	.R-...D- ..H-..j]z		
0210	68 34 94 71 98 37 15 79	65 0b e5 d1 9b 08 e6 79	h4.q.7.y e-....y		
0220	a5 8b 8f fc a5 cd 17 c1	33 4d b7 c4 cd dc 55 d0	3M-...U-		
0230	1a 5f 2d 90 03 d7 d6 c2	50 e3 ab dc 09 00 e0 25	.-.....P-....%		
0240	a1 ec 58 04 84 7b bd 4e	58 7c 6c 44 a9 16 aa 9f	..X-..{.N X 1D-...		
0250	36 39 54 74 17 b6 25 f5	8c af 88 61 e9 81 57 12	69Tt-..%- ...a-..W-		
0260	a0 a1 74 4b 29 9d 23 95	4b c2 77 ec 45 c5 9d c6	..tK)-.#- K-w-E-...		

SNI (Server Name Indication), позволяющее браузеру указать имя веб-сервера, с которым он пытается установить соединение. SNI необходимо для того, чтобы на одном IP-адресе могли корректно откликаться по HTTPS веб-

серверы, размещённые под разными доменами.

4) Согласно спецификации, после отправки ClientHello клиент ожидает ответа сервера. В ответ может прийти либо сообщение об ошибке, в виде Alert, либо сообщение ServerHello. Если сервер смог успешно обработать ClientHello, то он отвечает сообщением ServerHello. Это сообщение также имеет заголовок из четырёх байтов: тип сообщения (один байт) и длина (три байта). ServerHello содержит следующие поля:

4.1) Сообщение ServerHello

972	10:19:42,064570	192.168.19.254	178.248.237.68	TCP	1514	56390 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=1460 [TCP PDU reassembled in 978]
973	10:19:42,064570	192.168.19.254	178.248.237.68	TLSv1.3	344	Client Hello (SNI=habr.com)
974	10:19:42,079783	178.248.237.68	192.168.19.254	TCP	60	443 → 56390 [ACK] Seq=1 Ack=1751 Win=45260 Len=0
975	10:19:42,083112	178.248.237.68	192.168.19.254	TLSv1.3	1514	Server Hello, Change Cipher Spec, Application Data
976	10:19:42,084836	178.248.237.68	192.168.19.254	TCP	1514	443 → 56390 [ACK] Seq=1461 Ack=1751 Win=45260 Len=1460 [TCP PDU reassembled in 978]
977	10:19:42,084918	192.168.19.254	178.248.237.68	TCP	54	56390 → 443 [ACK] Seq=1751 Ack=2921 Win=64240 Len=0
978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3	617	Application Data, Application Data, Application Data
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	54	56390 → 443 [ACK] Seq=1751 Ack=3484 Win=63677 Len=0

> Frame 975: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF_{A9DF8D75-8F6C-417D-A0ED-60DB749E1894}, id 0

▼ Ethernet II, Src: SuperMicroCo_44:8d:cd (0c:c4:7a:44:8d:cd), Dst: BrilinkTechn_09:04:0c (00:01:0e:09:04:0c)

> Destination: BrilinkTechn_09:04:0c (00:01:0e:09:04:0c)

> Source: SuperMicroCo_44:8d:cd (0c:c4:7a:44:8d:cd)

Type: IPv4 (0x0800)

[Stream index: 28]

> Internet Protocol Version 4, Src: 178.248.237.68, Dst: 192.168.19.254

> Transmission Control Protocol, Src Port: 443, Dst Port: 56390, Seq: 1, Ack: 1751, Len: 1460

▼ Transport Layer Security

▼ TLSv1.3 Record Layer: Handshake Protocol: Server Hello

Content Type: Handshake (22)

Version: TLS 1.2 (0x0303)

Length: 122

0000 00 01 0e 09 04 0c 0c c4 7a 44 8d cd 08 00 45 00zD....E..

0010 05 dc 7e ea 40 00 39 06 48 4e b2 f8 ed 44 c0 a8@-9- HN...D..

4.2) Версия протокола, которую будут использовать клиент и сервер;

974	10:19:42,079783	178.248.237.68	192.168.19.254	TCP	60	443 → 56390 [ACK] Seq=1 Ack=1751 Win=45260 Len=0
975	10:19:42,083112	178.248.237.68	192.168.19.254	TLSv1.3	1514	Server Hello, Change Cipher Spec, Application Data
976	10:19:42,084836	178.248.237.68	192.168.19.254	TCP	1514	443 → 56390 [ACK] Seq=1461 Ack=1751 Win=45260 Len=1460 [TCP PDU reassembled in 978]
977	10:19:42,084918	192.168.19.254	178.248.237.68	TCP	54	56390 → 443 [ACK] Seq=1751 Ack=2921 Win=64240 Len=0
978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3	617	Application Data, Application Data, Application Data
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP	54	56390 → 443 [ACK] Seq=1751 Ack=3484 Win=63677 Len=0

▼ TLSv1.3 Record Layer: Handshake Protocol: Server Hello

Content Type: Handshake (22)

Version: TLS 1.2 (0x0303)

Length: 122

▼ Handshake Protocol: Server Hello

Handshake Type: Server Hello (2)

Length: 118

> Version: TLS 1.2 (0x0303)

Random: 72ba0a241c518a75d5bde6a698004868deb1c0a1a352a5d918249926c526178

Session ID Length: 32

Session ID: d3028f7e4d8fc3b5471bd3283a9e2a83e96b9afe67bd632566ec4457afd73434

Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)

Compression Method: null (0)

0030 b0 cc b3 9f 00 00 16 03 03 00 7a 02 00 00 76 03-z....V..

0040 03 72 ba 0a 24 1c 51 8a 75 d5 bd be 6a 69 80 04\$.Q- u---jL..

0050 86 8d eb 1c 0a 1a 35 2a 5d 91 82 49 02 6c 52 615\$- j---1B..

0060 1a 20 d3 02 8f 7e 4d 8f c3 b5 47 1b d3 28 3a 9eH- G- (---

0070 2a 83 e9 6b 9a fe 67 bd 63 25 66 ec 44 57 af d7k- g- cKf DM-..

0080 34 34 13 01 00 00 2e 00 2b 00 02 03 04 00 33 00 44.....+.....3-

0090 24 00 1d 00 20 b0 14 74 ca 5d f8 ec b6 36 73 92t-]---6s..

00a0 e2 c9 03 ef e2 0a 30 7d e2 fe 0d a5 74 71 28 2a0)tq(*

00b0 12 8d c9 7- 38 14 03 03 00 01 01 17 03 03 00 7a@.....t

4.3) 32 байта случайных значений - Server Random. С этой строкой ситуация такая же, как и с Client Random: первые четыре байта могут быть таймстемпом, а могут и не быть. В нашем случае это не таймстеп.

978	10:19:42,085651	178.248.237.68	192.168.19.254	TLSv1.3
980	10:19:42,133586	192.168.19.254	178.248.237.68	TCP
Length: 118 > Version: TLS 1.2 (0x0303) Random: 72ba0a241c518a75d5bde6a698004868deb1c0a1a352a5d918249926c526178 Session ID Length: 32 Session ID: d3028f7e4d8fc3b5471bd3283a9e2a83e96b9afe67bd632566ec4457afd73434 Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301) Compression Method: null (0) Extensions Length: 46 > Extension: supported_versions (len=2) TLS 1.3 > Extension: key_share (len=36) x25519 [JA3S Fullstring: 771,4865,43-51] [JA3S: f4febc55ea12b31ae17cfb7e614afda8]				
▼ TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec				
0080	34 34 13 01 00 00 2e 00	2b 00 02 03 04 00 33 00	44.....+.....3.	
0090	24 00 1d 00 20 b0 14 74	ca 5d f8 ec b6 36 73 92	\$... ..t .]...6s.	
00a0	e2 c9 03 ef e2 0a 30 7d	e2 fc 0d e5 74 71 28 2a	}... ..tq(*	
00b0	12 8d c9 2c 38 14 03 03	00 01 01 17 03 03 00 24	... ,8...	
00c0	da f9 ac 88 a6 57 d3 27	6b 40 1a 81 89 bf 95 3a	W.' k@.....:	
00d0	0d 99 4e 51 e6 69 0b 7b	e6 6f b6 05 3b 9d 2e 44	..NQ.i.{ ..o...;.D	
00e0	68 5d 0c 94 17 03 03 0c	49 2d 63 40 6d 89 ca 6c	h]... ..I-c@m..l	
00f0	34 27 e5 8b 1c e2 76 27	22 99 33 cc 26 bd af 45	4'.....v' ".3.&..E	
0100	a3 12 b4 6b b6 85 0f 0c	d1 0c d1 fa 8b 05 ed 4e	...k..... ..N	
0110	3d df cf f7 53 21 60 37	2c 1c 0f 64 56 55 49 20	=...S!`7 ,..dVUI	
0120	e5 81 87 49 98 4b 0a 29	9f 14 c2 53 2d 19 90 19	...I.K.) ...S-...	
0130	2d 0e b6 fd 6f 91 c8 12	60 2f fb f6 17 97 0c 26	...o...`/.....&	
0140	b2 f1 9e 22 f9 01 76 92	0c 72 ff 9e 00 9e 3d 22	...".v. .r.....="	
0150	48 0d 09 a4 9a 42 7a 32	a4 ff b2 32 41 9c 53 ce	H...Bz2 ...2A.S.	
0160	58 e2 5a ae a7 8b 38 8b	27 b4 8d e0 b3 9e 89 ce	X-Z...8. '.....	

4.6) Выбранный сервером метод сжатия — это null;

1021	10:19:42,247308	178.248.237.68	192.168.19.254	TCP	60 443 → 56391 [ACK] Seq=1719 Win=45260 Len=0
1025	10:19:42,251984	178.248.237.68	192.168.19.254	TLSv1.3	1514 Server Hello, Change Cipher Spec, Application Data
1026	10:19:42,254272	178.248.237.68	192.168.19.254	TCP	1514 443 → 56391 [ACK] Seq=1461 Ack=1719 Win=45260 Len=1460 [TCP PDU reassembled in 1028]
1027	10:19:42,254368	192.168.19.254	178.248.237.68	TCP	54 56391 → 443 [ACK] Seq=1719 Ack=2921 Win=64240 Len=0
1028	10:19:42,255020	178.248.237.68	192.168.19.254	TLSv1.3	618 Application Data, Application Data, Application Data
Length: 1709 > Version: TLS 1.2 (0x0303) Random: 8c2a0d3068df6fe6aee8aa72aa487515bac0c5c39664a70976b8b447cf9df0d0 Session ID Length: 32 Session ID: d3aae2cad422049fc938510bedf22c9f4e8fbbae1d550480d8f24aaec2de5b7 Cipher Suites Length: 32 > Cipher Suites (16 suites) Compression Methods Length: 1 > Compression Methods (1 method) Compression Method: null (0) Extensions Length: 1604 > Extension: Reserved (GREASE) (len=0) > Extension: supported_groups (len=12)					
0060	cc a8 c0 13 c0 14 00 9c	00 9d 00 2f 00 35 01	/.5.		
0070	06 44 0a 0a 00 00 0a 0a	00 0c 00 0a fa fa 11 ec	..D.....		
0080	00 1d 00 17 00 18 ff 01	00 01 00 44 69 00 05 00	Di...		
0090	03 02 68 32 00 05 00 05	01 00 00 00 00 00 00 00	..h2.....		
00a0	0d 00 0b 00 00 08 68 61	62 72 2e 63 6f 6d 00 0b	ha br.com		
00b0	00 02 01 00 00 2b 00 07	06 1a 1a 03 04 03 03 00			
00c0	12 00 00 00 10 00 0e 00	0c 02 68 32 08 68 74 74	h2-htt		
00d0	70 2f 31 2e 11 00 2d 00	02 01 01 00 33 04 ef 04	p/1.1... ..3...		
00e0	ed fa fa 00 01 00 11 ec	04 c0 9f 55 a4 e1 32 4d	U..2M		
00f0	51 d1 2c 10 dc 2b d5 68	3a 81 20 ca 8b 0a 77 29	Q.,...h :...w)		
0100	1a 70 c8 7b ba 0f 4a 8f	65 ea 94 aa 42 c1 98 09	.p.(...J. e...B...		
0110	46 3f 61 55 da ea 5b 3b	b0 8e 48 0b c7 f2 f1 93	F?aU...[; ..H.....		
0120	de 05 c1 76 6a 60 7c e4	1e 66 c1 c5 21 ba 12 05	...vj' ...f...l...		

4.7) Некоторый набор расширений, который поддерживает сервер.

```

1003 10:19:42,2317173 192.168.19.254      178.248.237.68      TCP          1514 56391 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=1460 [TCP PDU reassembled in 1004]
1004 10:19:42,2317173 192.168.19.254      178.248.237.68      TLSv1.3      312 Client Hello (SNI=habr.com)
1021 10:19:42,247308 178.248.237.68      192.168.19.254      TCP          60 443 → 56391 [ACK] Seq=1 Ack=1719 Win=45260 Len=0
1025 10:19:42,251984 178.248.237.68      192.168.19.254      TLSv1.3      1514 Server Hello, Change Cipher Spec, Application Data
1026 10:19:42,254272 178.248.237.68      192.168.19.254      TCP          1514 443 → 56391 [ACK] Seq=1461 Ack=1719 Win=45260 Len=1460 [TCP PDU reassembled in 1028]
1027 10:19:42,254368 192.168.19.254      178.248.237.68      TCP          54 56391 → 443 [ACK] Seq=1719 Ack=2921 Win=64240 Len=0
1028 10:19:42,255020 178.248.237.68      192.168.19.254      TLSv1.3      618 Application Data, Application Data, Application Data
> Cipher Suites (16 suites)
  Compression Methods Length: 1
  Compression Methods (1 method)
    Compression Method: null (0)
  Extensions Length: 1604
> Extension: Reserved (GREASE) (len=0)
> Extension: supported_groups (len=12)
> Extension: renegotiation_info (len=1)
> Extension: application_settings (len=5)
> Extension: status_request (len=5)
> Extension: server_name (len=13) name=habr.com
> Extension: ec_point_formats (len=2)
> Extension: supported_versions (len=7) TLS 1.3, TLS 1.2
0070 00 4d 0a 0a 00 00 00 0a 00 0c 00 0a fa fa 11 ec 41.....
0080 00 1d 00 17 00 18 ff 01 00 01 00 4a 69 00 05 00 .....Di...
0090 03 02 68 32 00 05 00 05 01 00 00 00 00 00 00 .....h2....
00a0 0d 00 0b 00 00 08 68 61 62 72 2e 63 6f 6d 00 0b .....habr.com...
00b0 00 02 00 00 2b 1b 00 07 00 1a 03 04 03 03 00 .....
00c0 12 00 00 00 19 00 0e 00 0c 02 68 32 00 68 74 74 .....h2-htt
00d0 70 2f 31 2e 31 00 2d 00 02 01 01 00 33 04 ef 04 p/1.1...3...
00e0 ed fa fa 00 01 00 11 ec 04 c0 9f 55 a4 e1 32 4d .....U..2M
00f0 51 d1 2c 10 dc 2b 45 68 3a 81 20 ca 8b 0a 77 29 Q...h...w)
0100 1a 70 c5 7b 0a 0f 4a 8f 05 ea 94 aa 42 c1 98 09 p[-?]-e...B...
0110 46 3f 61 55 da ea 5b 3b 80 8e 48 0b c7 f2 f1 93 FfAu[-]-H...
0120 de 05 c1 76 6a 60 7c 4e 1e 66 c1 c5 21 ba 12 05 ...v][-f..l...
0130 3c 4c 4f 66 74 e7 44 0f ef a2 9b 58 26 14 f3 46 c[-f]D...X&..F

```

К примеру SessionTicket TLS, показывает, что сервер поддерживает технологию выдачи тикетов сессии для клиента для последующего быстрого восстановления соединения TLS сессии.

В современных реализациях TLS раздел "Расширения" (Extensions) имеет очень большое значение, так как в нём передаются параметры, определяющие схему работы и клиента, и сервера. Предназначение ServerHello - согласовать параметры соединения.

5) За отправкой ServerHello, со стороны сервера следует ряд других сообщений, состав и содержание этих сообщений зависят от выбранного сервером режима работы.

5.1) Certificate. Ключевым аспектом для современных реализаций TLS является использование SSL-сертификатов. Сертификаты передаются сервером в сообщении Certificate. Это сообщение присутствует практически всегда. Серверный сертификат содержит открытый ключ сервера. В теории, спецификация позволяет установить соединение без отправки серверных сертификатов. Это так называемый "анонимный" режим, однако он практически не используется, как и режимы TLS без шифрования. Так, примерно 100% веб-серверов, поддерживающих TLS, передают SSL-сертификаты. Для типичной конфигурации сообщение Certificate будет включать несколько SSL-сертификатов, среди которых один серверный сертификат и так называемые "промежуточные сертификаты", позволяющие клиенту выстроить цепочку валидации. Спецификация предписывает строгий

порядок следования сертификатов в сообщении: первым должен идти серверный сертификат, а последующие - в порядке удостоверения предыдущего. Однако на практике серверы нередко возвращают сертификаты в произвольном порядке.

Действительно далее идет пакет с сертификатом

Wireshark packet capture showing a TLS handshake. The packet list shows a 'Certificate' packet (No. 1987) with a length of 1354 bytes. The packet details show 'Length: 78', 'Version: TLS 1.2 (0x0303)', and 'Random'. The packet bytes show the start of the certificate structure: 'T...f... 00 00 00 ff 01 00 01 00'.

5.2) Сообщение Certificate (тип – 0x0b=11)

Handshake Protocol: Certificate

Handshake Type: Certificate (11)

Length: 4567

Certificates Length: 4564

Certificates (4564 bytes)

Certificate Length: 1607

Certificate: 308206433082052ba003020102021013cf6c7bd39aecd37a... (:

signedCertificate

version: v3 (2)

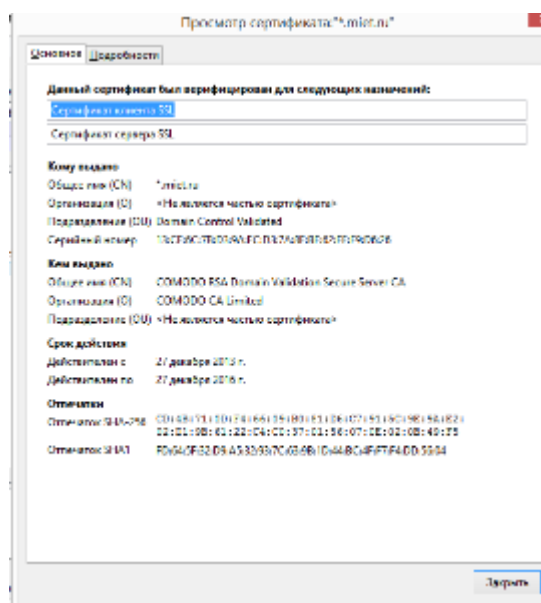
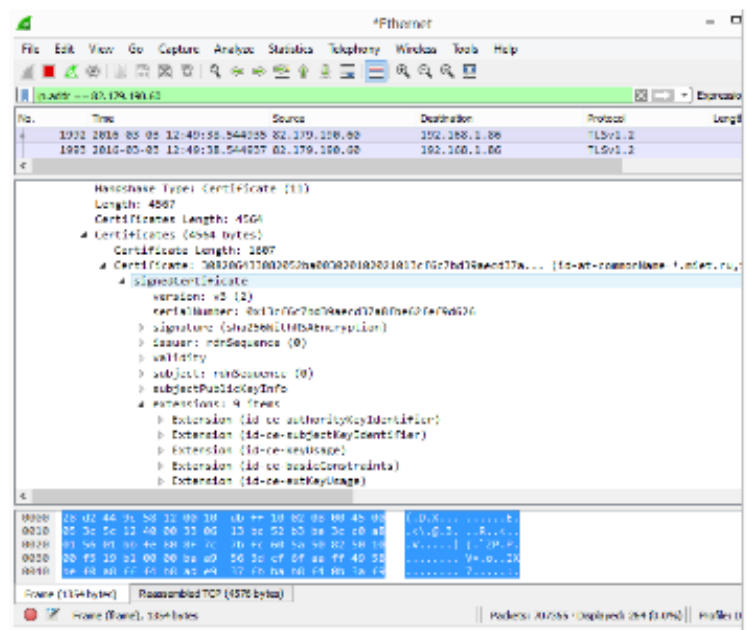
serialNumber: 0x13cf6c7bd39aecd37a8f6e62fe9d626

0000 16 03 03 11 db 0b 00 11 d7 00 11 d4 00 06 47 30G0

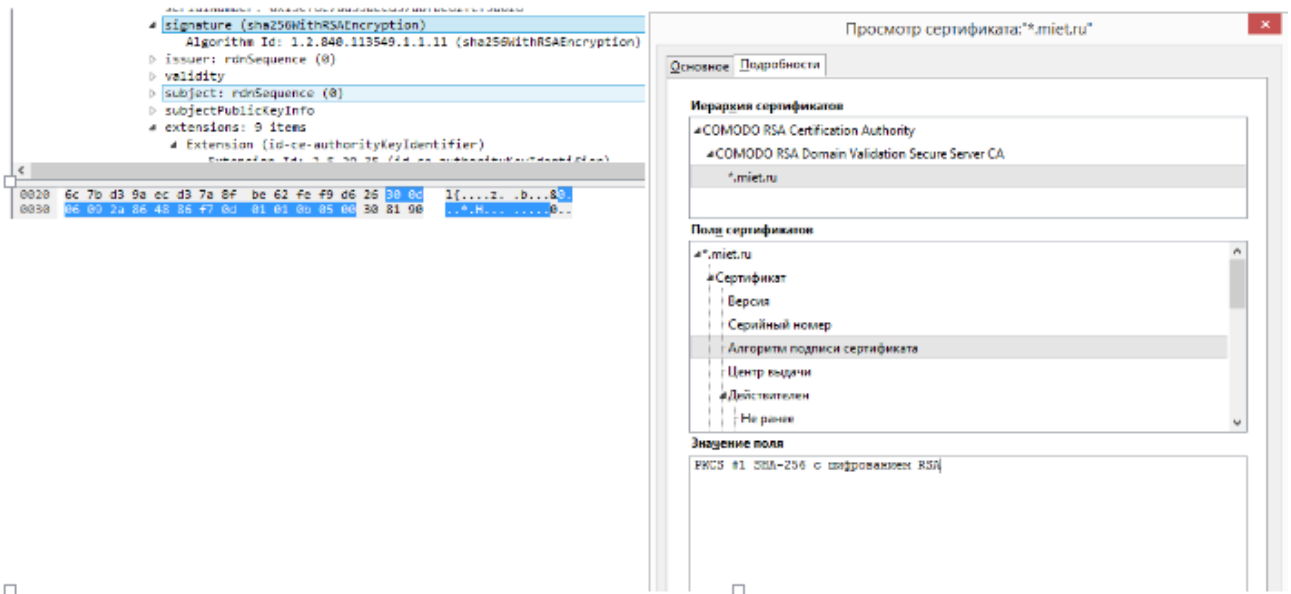
5.3) Длина сообщения (0x0011d7 = 4567)

Length: 4567
Handshake Protocol: Certificate
Handshake Type: Certificate (11)
Length: 4567
Certificates Length: 4564
- Certificates (4564 bytes) - Certificate Length: 1607 - Certificate: 308206433082052ba003020102021013cf6c7bd39aecdd37a... (id - signedCertificate - version: v3 (2) - serialNumber: 0x13cf6c7bd39aecdd37a8fbe62fef9d626
000 16 03 03 11 db 0b 00 11 d7 00 11 d4 00 06 47 30G0

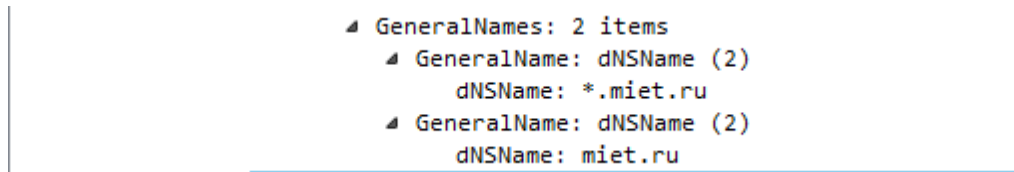
5.4) Данные отображаемые в Wireshark и информация о сертификате из браузера. Как видим серийные номера совпадают.



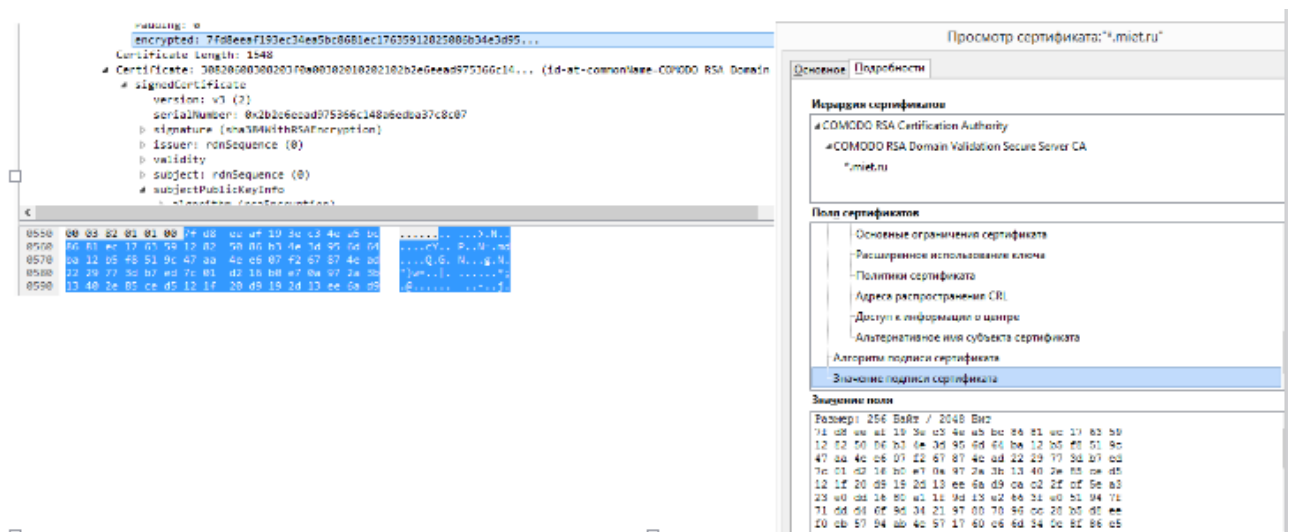
5.5) Алгоритм подписи сертификата.



5.6) Разрешенные DNS имена, на которые действует сертификат



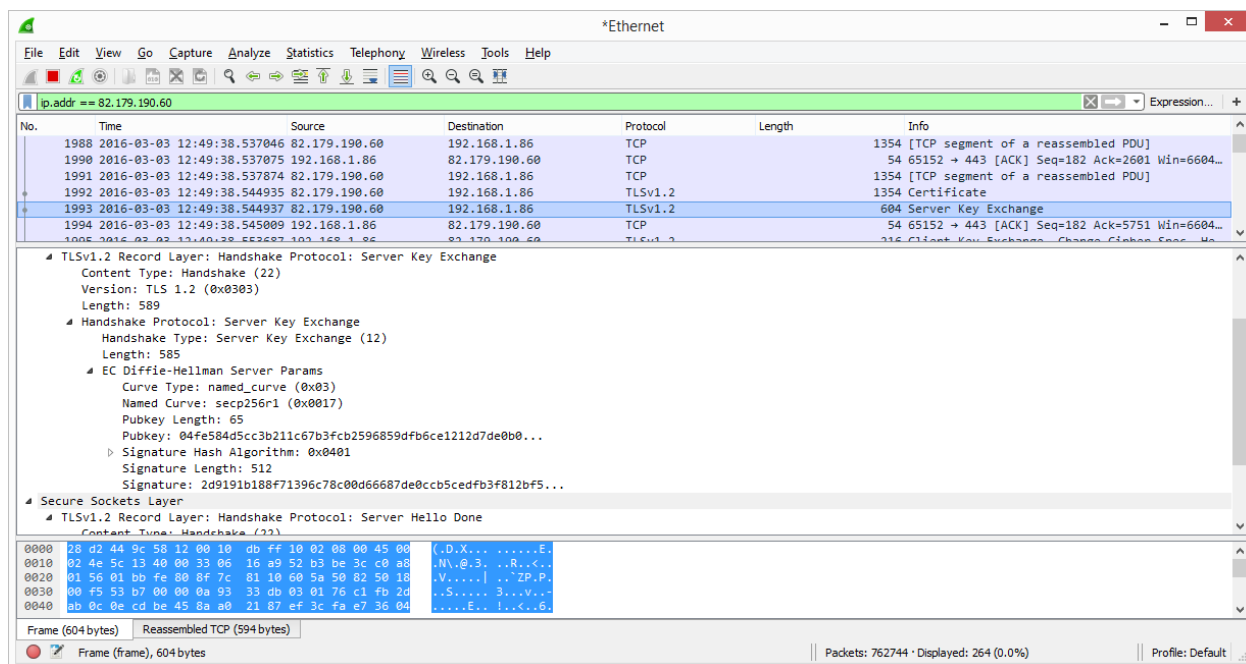
5.7) Значение подписи сертификата. Подпись вычисляется от значения хеш-функции, аргументом которой является весь сертификат (за исключением, соответственно, подписи). Генерируется подпись при помощи секретного ключа УЦ, соответствующего сертификату с именем, указанным в Issuer.



Следующий шаг при установке безопасного соединения — это отправка

параметров для генерации общего сеансового ключа.

6) ServerKeyExchange. Это сообщение, содержащее серверную часть данных, необходимых для генерации общего сеансового ключа. Это сообщение может отсутствовать. В зависимости от выбранного шифронабора, данных, содержащихся в SSL-сертификате, может быть недостаточно для выработки общего ключа. Недостающие данные как раз и передаются в ServerKeyExchange. Обычно это параметры протокола Диффи-Хеллмана (DH). Если используется классический вариант, то в сообщении ServerKeyExchange передаётся значение модуля и вычисленный сервером открытый ключ DH. В варианте на эллиптических кривых (ECDH) - идентификатор самой кривой и, аналогично DH, открытый ключ. Параметры подписываются сервером, клиент может проверить подпись, используя открытый ключ сервера из SSL-сертификата. В зависимости от используемой криптосистемы подпись может быть DSA (сейчас практически не встречается), RSA или ECDSA.



6.1) Сообщение ServerKeyExchange (тип - 0x0c = 12), длина данных (0x000249 = 585 байтов). Так как выбран шифронабор с ECDHE, то сообщение ServerKeyExchange содержит параметры именно для алгоритма ECDHE. Каких-то специальных флагов, обозначающих ServerKeyExchange как ECDHE, - не предусмотрено, всё определяется шифронабором.

- Handshake Protocol: Server Key Exchange
 - Handshake Type: Server Key Exchange (12)
 - Length: 585
- EC Diffie-Hellman Server Params
 - Curve Type: named_curve (0x03)
 - Named Curve: secp256r1 (0x0017)
 - Pubkey Length: 65
 - Pubkey: 04fe584d5cc3b211c67b3fcb2596859dfb6ce1212d7de0b0...
 - Signature Hash Algorithm: 0x0401
 - Signature Length: 512
 - Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...

Secure Sockets Layer

- TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
 - Content Type: Handshake (22)

000	16 03 03 02 4d 0c 00 02 49 03 00 17 41 04 fe 58	M...I...A..X
-----	---	------------------

6.2) Тип кривой (0x03 - типовая именованная кривая).

- EC Diffie-Hellman Server Params
 - Curve Type: named_curve (0x03)
 - Named Curve: secp256r1 (0x0017)
 - Pubkey Length: 65
 - Pubkey: 04fe584d5cc3b211c67b3fcb2596859dfb6ce1212d7de0b0...
 - Signature Hash Algorithm: 0x0401
 - Signature Length: 512
 - Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...

Secure Sockets Layer

- TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
 - Content Type: Handshake (22)
 - Version: TLS 1.2 (0x0303)
 - Length: 4

000	16 03 03 02 4d 0c 00 02 49 03 00 17 41 04 fe 58	M...I...A..X
-----	---	------------------

6.3) Идентификатор выбранной сервером кривой (0x0017 - secp521r1, реестр кривых ведёт IANA, каждая из типовых кривых содержит реестр кривых ведёт IANA, каждая из типовых кривых содержит в своём определении необходимые для ДН параметры, которые строго зафиксированы: это генератор, разрядность группы точек).

- EC Diffie-Hellman Server Params
 - Curve Type: named_curve (0x03)
 - Named Curve: secp256r1 (0x0017)
 - Pubkey Length: 65
 - Pubkey: 04fe584d5cc3b211c67b3fcb2596859dfb6ce1212d7de0b0...
 - Signature Hash Algorithm: 0x0401
 - Signature Length: 512
 - Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...

Secure Sockets Layer

- TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
 - Content Type: Handshake (22)

0000	16 03 03 02 4d 0c 00 02 49 03 00 17 41 04 fe 58	M...I...A..X
------	---	------------------

6.4) Длина поля публичного ключа, которое идёт следом (0x41 = 65 байта)

```

    EC Diffie-Hellman Server Params
      Curve Type: named_curve (0x03)
      Named Curve: secp256r1 (0x0017)
      Pubkey Length: 65
      Pubkey: 04fe584d5cc3b211c67b3fcb2596859dfb6ce1212d7de0b0...
      Signature Hash Algorithm: 0x0401
      Signature Length: 512
      Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...
  Secure Sockets Layer
    TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
      Content Type: Handshake (22)
      Version: TLS 1.2 (0x0303)
      Length: 4
    Handshake Protocol: Server Hello Done
0000 16 03 03 02 4d 0c 00 02 49 03 00 17 41 04 fe 58 ....M... I...A..X

```

6.5) 65 байта, представляющие собой публичный ключ ECDHE сервера. Публичный ключ — это точка на кривой, которую сервер вычислил, используя параметры кривой. В соответствии с форматом записи точек, указанным клиентом в расширении ClientHello, передаются аффинные координаты точки (x, y) (запись предваряется заголовком). Клиенту параметры кривой известны, потому что используется типовая кривая.

```

      Pubkey Length: 65
      Pubkey: 04fe584d5cc3b211c67b3fcb2596859dfb6ce1212d7de0b0...
      Signature Hash Algorithm: 0x0401
      Signature Length: 512
      Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...
  Secure Sockets Layer
    TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
      Content Type: Handshake (22)
      Version: TLS 1.2 (0x0303)
      Length: 4
    Handshake Protocol: Server Hello Done
      Handshake Type: Server Hello Done (14)
      Length: 0
0000 16 03 03 02 4d 0c 00 02 49 03 00 17 41 04 fe 58 ....M... I...A..X
0010 4d 5c c3 b2 11 c6 7b 3f cb 25 96 85 9d fb 6c e1 M\....{? .%....1.
0020 21 2d 7d e0 b0 28 33 96 af 00 c5 7e e0 99 0b ea !-}..(3. ...~....
0030 1b 9a d4 60 f7 0a 93 33 db 03 01 76 c1 fb 2d ab ...^...3 ...v...-
0040 0c 0e cd be 45 8a a0 21 87 ef 3c fa e7 36 04 01 ....E..! ..<..6..

```

6.6) Длина подписи, соответствующей параметрам DH (0x0100 - 256 байтов). За значением открытого ключа DH следует серверная подпись, выполненная по алгоритму и с ключом, соответствующим указанным в серверном сертификате (в нашем случае это RSA).

```

Signature Length: 512
Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...
Secure Sockets Layer
  TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 4
    Handshake Protocol: Server Hello Done
      Handshake Type: Server Hello Done (14)
      Length: 0
0050 02 00 2d 91 91 b1 88 f7 13 96 c7 8c 00 d6 66 87 ..-.....f.

```

6.7) Значение подписи DH.

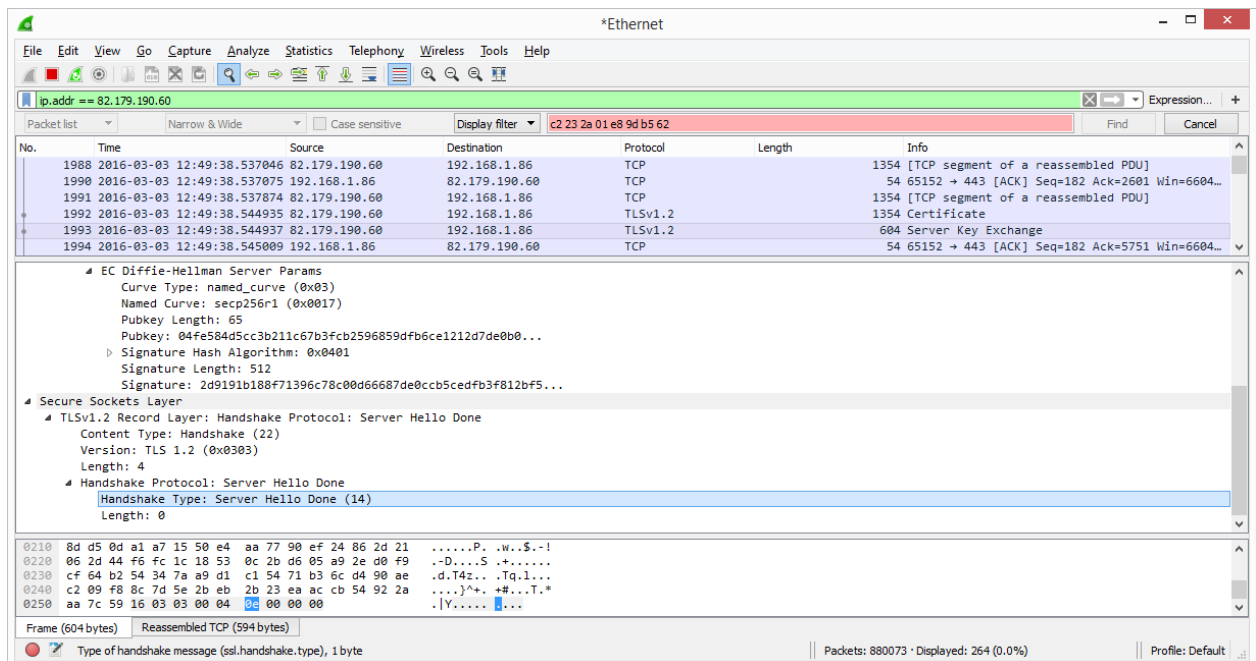
```

Signature: 2d9191b188f71396c78c00d66687de0ccb5cedfb3f812bf5...
Secure Sockets Layer
  TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 4
    Handshake Protocol: Server Hello Done
      Handshake Type: Server Hello Done (14)
      Length: 0
0050 02 00 2d 91 91 b1 88 f7 13 96 c7 8c 00 d6 66 87 ..-.....f.
0060 de 0c cb 5c ed fb 3f 81 2b f5 fe d1 1f a7 9b 18 ... \..?. +.....
0070 ba f7 43 80 f7 5f a2 10 a5 22 ed 74 44 0e b5 e9 ..C._. ".tD...
0080 93 46 56 e7 28 d8 0f 2c 1a ce ad 85 56 8e 7b c3 .FV.(., ....V.{.
0090 72 02 2c 57 c4 9a 06 1f 4d 8b 25 5e 3d cc c9 ef r.,W.... M.%^=...

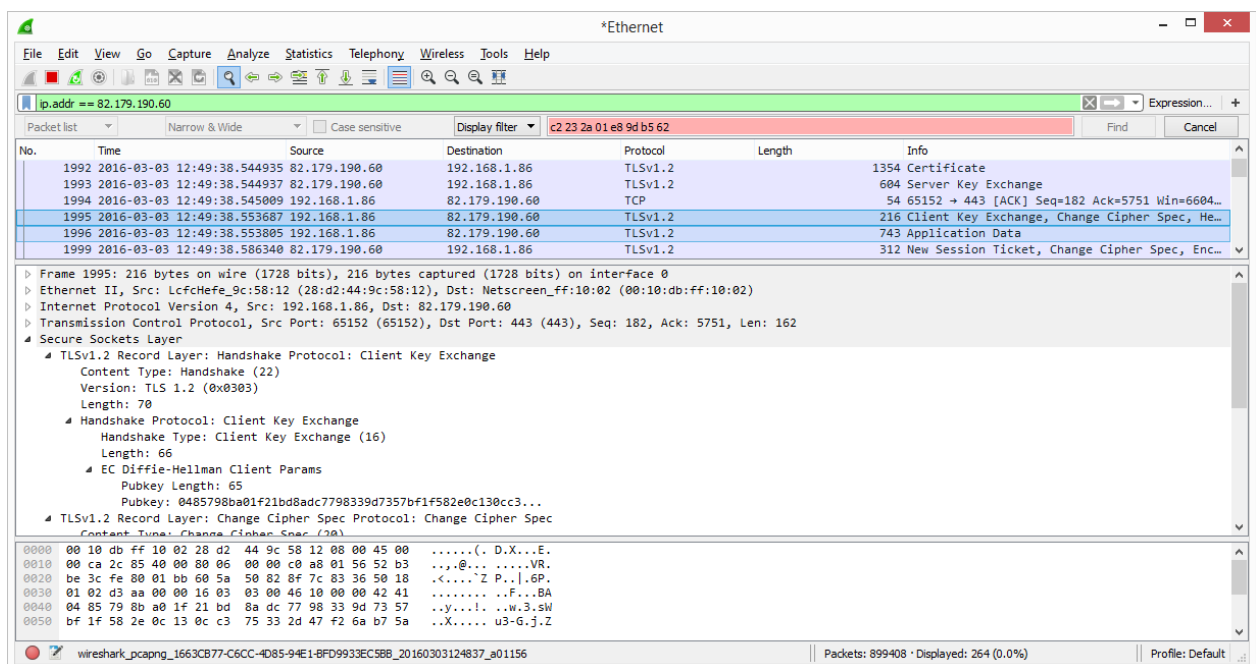
```

7) ServerHelloDone - обозначает окончание пакета сообщений, возглавляемого ServerHello. Это сообщение имеет нулевую длину (однако заголовок никто не отменял, поэтому в TLS-записи ServerHelloDone соответствует 4 байта) и служит простым флагом, обозначающим, что сервер передал свою часть начальных данных и теперь ожидает ответа от клиента.

Сообщение ServerHelloDone (тип - 0x0e = 14). Это сообщение имеет нулевую длину, поэтому следом за типом идут три байта с нулевыми значениями (это обозначающие длину байты).



8) Следующий шаг установки TLS соединения ClientKeyExchange



ClientKeyExchange - клиентская часть обмена данными, позволяющими узлам получить общий сеансовый. В этом случае, ClientKeyExchange содержит открытый ключ ДН. Этот ключ генерируется клиентом либо в соответствии с параметрами, переданными сервером в ServerKeyExchange, либо в соответствии с параметрами, указанными в серверном SSL-сертификате, если последний поддерживает ДН ключ.

Собственно сам ключ.

содержит отпечаток - значение хеш-функции, - от всех предыдущих сообщений Handshake, отправленных, на данный момент, и клиентом, и сервером. Получив это сообщение в защищённой TLS-записи, сервер может вычислить хеш-сумму от известных ему предшествовавших сообщений и сравнить результат. Таким образом подтверждается подлинность сообщений и, соответственно, оказываются криптографически защищены выбранный шифронабор и другие параметры соединения.

Задание к практической работе

Порядок выполнения практической работы:

- 1) Изучить информацию, приведенную в теоретической части.
- 2) Проверить установку и при необходимости установить Wireshark и Firefox на компьютер.
- 3) Выбрать не менее 5-ти веб-серверов различной организационной и государственной принадлежности.
- 4) Выбрать исследуемый веб-сервер
- 5) Запустить Wireshark и используя Firefox установить https соединение с выбранным сервером.
- 6) Установить Wireshark и провести анализ соединения как это описано в разделе 4
- 7) Сохранить данные необходимы для последующего сравнительного анализа:
 - a) Имя сервера, его характеристики
 - b) Версия TLS
 - c) Выбранные алгоритмы шифрования
 - d) Полученный сертификат: версия. валидность сертификата, валидность ключа, удостоверяющий центр.
 - e) Время установки соединения (от ClientHello до Finisfed)
- 8) Если список исследуемых серверов не исчерпан выбрать другой

сервер и вернуться к пункту 3)

9) Если браузер поддерживал соединение TLS 1.2 принудительно изменить параметры TLS соединения в Firefox на TLS 1.0 (в браузере перейти по “about:config” и изменить раздел SSL\TLS) и перейти к пункту 4).

10) Провести сравнительный анализ полученной информации.

11) В качестве отчета представить результаты сравнительного анализа, выводы в отношении безопасности и корректности настройки веб-серверов с учетом их организационной и государственной принадлежности.

Составьте отчет о выполнении практической работы.

Включите в него копии экрана пошагового выполнения задания и ответы на вопросы практической работы.